

Cross-Platform Mobile App Development

Week 1 — The Mobile Development Landscape (2026)

Instructor: Nguyen Thanh Tuan, PhD



PWA

Today's agenda

1. Market numbers 2026 — why cross-platform is the default, not the exception
2. Compare 4 strategies: Native / Hybrid (Capacitor) / Cross-platform (Flutter, React Native) / PWA
3. 4 real case studies: Spotify, Alibaba, Discord, Burger King
4. Get oriented on 3-layer architecture + state management
5. Meet the 3 mini-projects for the semester + the Capstone
6. Preview the 15-week learning path

Part 1: Market Trends

Cross-platform is the default now, not the "budget option"

Comparison	Native	Cross-Platform
Team	2 teams (iOS + Android)	1 team
Time to MVP	6–8 months	3–4 months
Annual cost	\$200K–300K	\$100K–150K
Code reuse	0%	90–95% (business logic)

Modern cross-platform performance: 85–95% of native — the performance gap is rarely the deciding factor anymore.

5 trends shaping the industry (2026)

- 🏠²⁴ **Super Apps** — Grab, WeChat, Zalo: bundling many services into one app
- 🤖 **AI Everywhere** — on-device ML vs. cloud AI, each with its own trade-offs
- 📱 **Cross-platform goes mainstream** — no longer a "stopgap" solution
- ⚡ **Low-code/No-code** — FlutterFlow, Bubble: great for prototyping, not for complex production apps
- 📶 **Offline-first & PWA** — gaining serious ground in poor-connectivity markets

AI trade-off: on-device vs. cloud

	On-device ML	Cloud AI
Privacy	✓ High	⚠ Data leaves device
Offline	✓ Works	✗ Needs connectivity
API cost	✓ None	✗ Pay per request
Power/freshness	⚠ Limited model	✓ Powerful, always current

→ Week 14 (AI in Mobile Apps) revisits this when we integrate an AI feature into the Capstone.

Why offline-first isn't a "nice to have"

Google's data:

- **53%** of users abandon a page that takes more than 3 seconds to load
- **70%** of users in emerging markets have unstable connectivity
- PWAs can increase engagement by up to **136%** (AliExpress case study)

→ This is why offline-first architecture is a CORE skill in this course (Chapters 2–3), not an advanced extra.

Job market snapshot

- **React Native**: high demand, large community, easy to hire for
- **Flutter**: fastest-growing (+15%/year)
- **Native iOS/Android**: still stable but growth is flattening

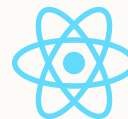
Advice: beginners often start with React Native (JS skills transfer easily); Flutter suits developers who want to specialize in mobile; Native is for apps that demand maximum performance.

Part 2: 4 Mobile Development Strategies

Native vs. Hybrid vs. Cross-Platform vs. PWA

Criterion	Native	Hybrid (Capacitor)	Cross-Platform	PWA
Language	Swift/Kotlin	Existing HTML/CSS/JS	Dart / JS-TS	HTML/CSS/JS
Performance	100%	60–70%	85–95%	70–80%
Hardware access	Full	Good (via plugins)	~90% via plugins	Very limited
App store	✓	✓ (a real native build)	✓	✗ (unless wrapped)
Best for	AAA games, AR/VR	Already have a PWA, need to ship fast	Most business apps	Content sites, MVPs

PWA



Decision Framework — which technology?

1. **Native** — AR/VR, heavy games, large team, max performance is priority #1
2. **Hybrid (Capacitor)** — already have a PWA/web app, need to ship to app stores fast, team only knows web
3. **Cross-platform (Flutter/RN)** — most business apps, need both stores, medium team/budget
4. **PWA** — no app store needed, poor-connectivity market, fastest time-to-market matters most

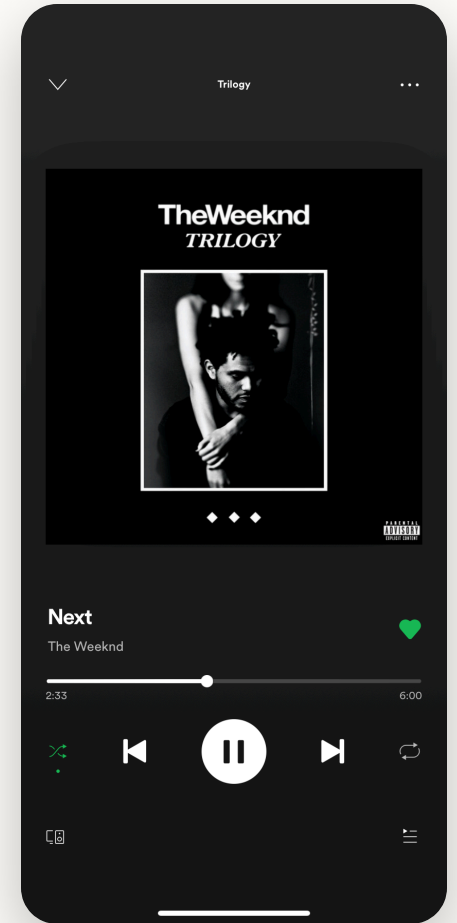
This is exactly the logic we'll use to choose a stack for each mini-project in this course.

Part 3: 4 Real-World Case Studies



Spotify (PWA)

- **Challenge:** music streaming needs fast loading + offline playback
- **Solution:** a PWA with caching strategies + Service Workers
- **Result:** 30% faster on mobile, offline mode for premium users

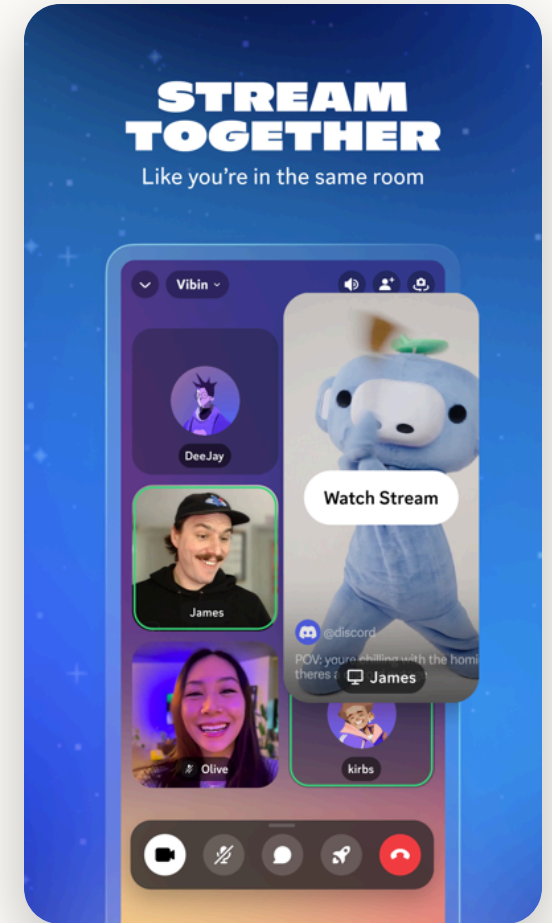


Alibaba (Flutter)

- **Challenge:** unify iOS + Android teams, cut duplicated work
- **Solution:** migrated the Xianyu app (500M+ users) to Flutter
- **Result:** 50% reduction in development time, 90% code reuse, native-comparable performance (60fps)

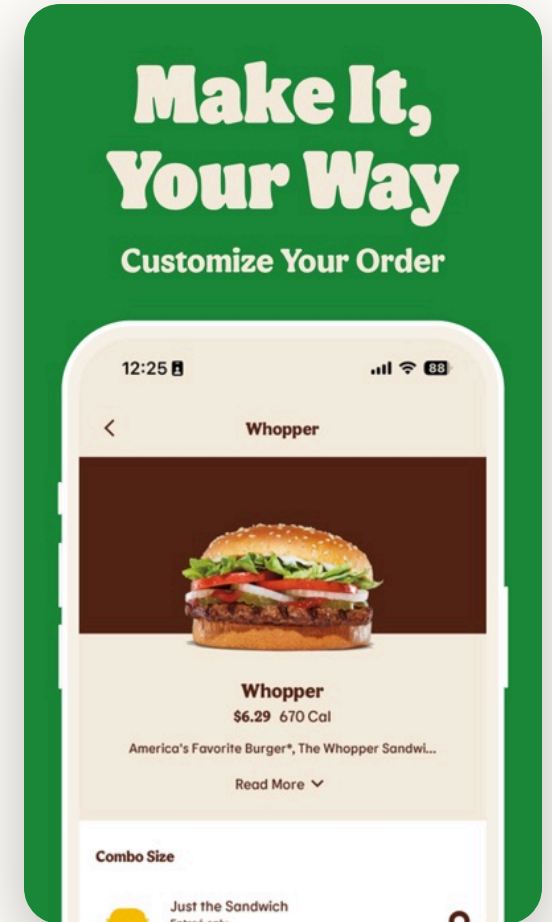
Discord (React Native)

- **Challenge:** fast iteration, cross-platform feature parity
- **Solution:** React Native for the core app
- **Result:** 95% code reuse, weekly release cadence



Burger King (Capacitor)

- **Challenge:** a strict brand design system had to stay consistent across web, TV ads, in-store menus, and mobile
- **Solution:** Capacitor — bring the existing web UI library straight to mobile, no native rebuild
- **Result:** app feels fully native, while every visual element stays pixel-consistent with the brand's design system



Part 4: Architecture & State Management (kickoff)

Why good architecture matters from day one

Unstructured code:

- ✗ Business logic tangled with UI
- ✗ Logic can't be tested in isolation
- ✗ Hard to maintain as the app grows
- ✗ Teamwork gets difficult

Solution: split into 3 layers — Presentation → Business Logic → Data
(dependencies flow one way only, see next slide)

The three-layer split, visually

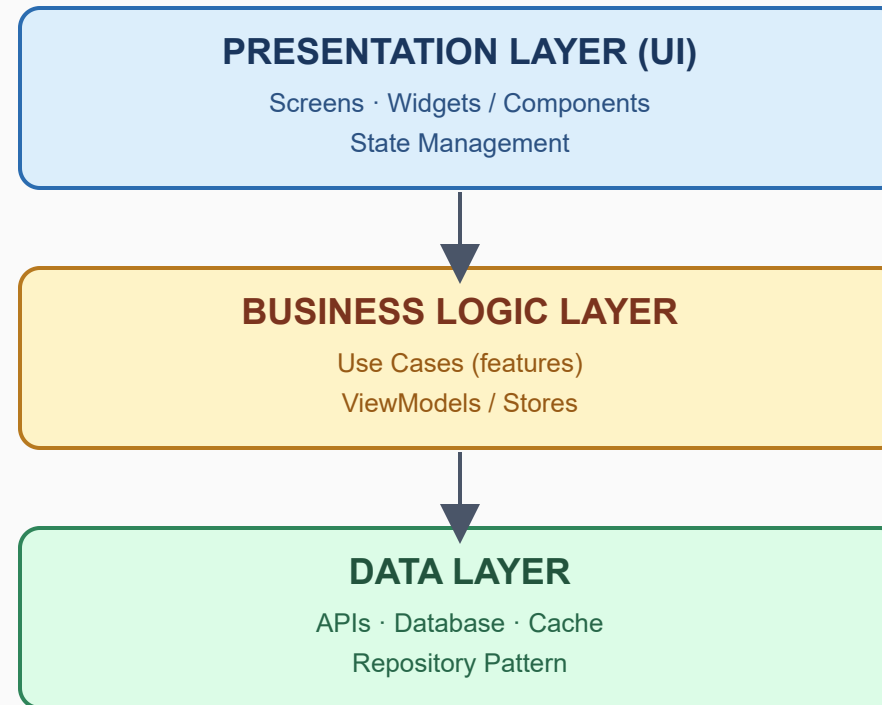


Figure 1.1 — The three-layer separation that keeps UI, logic, and data independently testable

What is state, and when do you need a library for it?

Screen count	Complexity	Suggested solution
1–3 screens	Simple	useState / setState
3–10 screens	Medium	Provider, Context API, Zustand
10+ screens	Complex	Riverpod, Redux Toolkit, BLoC

→ Week 6-8 (React Native) and Week 9-11 (Flutter) go deep on **two separate tracks** — not one merged timeline.

Part 5: 3 Mini-Projects + Capstone

3 mini-projects across the semester

Project	Stack	Modeled after
VKU Field Survey	PWA → Capacitor (Ch2–3)	ODK Collect (2M users, 250M submissions/year)
Room Booking	React Native (Ch4)	Robin / YAROOMS / LibCal
Expense OCR	Flutter (Ch5)	Smart Receipts / Money Lover

Why not a To-Do app or Weather app? Because an AI coding assistant one-shots those in a single prompt — they don't build the skills that matter (dynamic form engines, race conditions when two clients write at once, custom OCR parsers).

Capstone project — anti-AI-one-shot policy



Rules:

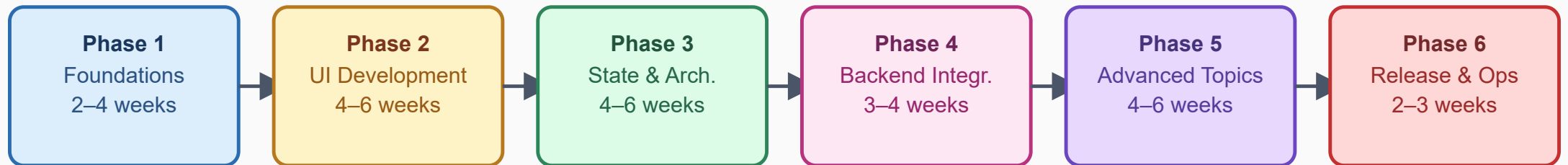
- Topic must be registered & approved before **Week 6**
- **Not accepted:** popular tutorial apps or near-copies of the 3 mini-projects already covered
- 4 starting-point suggestions: E-Commerce, Social Feed, Weather Dashboard, AI Note Helper — customize the domain/features so it doesn't match an existing tutorial
- **Must integrate ≥ 1 AI feature** with genuine practical purpose

Part 6: 15-Week Learning Path

Roadmap preview

Week	Content
1–2	Overview + Environment & UI/UX basics
3–4	PWA (Project 1)
5	Capacitor
6–8	React Native (Project 2)
9–11	Flutter (Project 3)
12	Advanced architecture & state
13	Backend & services
14	AI in mobile apps
15	Release & operations + Capstone defense

The six-phase path, visually



Total: 19–26 weeks full-time · 38–52 weeks part-time (2h/day)

Figure 1.2 — Recommended learning path across six phases

Today's takeaways



Cross-platform is the industry **default**, not a fallback choice



4 strategies, each with its **own sweet spot** — there's no single "best" answer



Good architecture & state management are **core skills**, taught from week one



The 3 mini-projects + Capstone are deliberately designed to **resist AI one-shotting**

Before Week 2: install Android Studio, VS Code, Git — full checklist coming by email.

Questions & homework

Come to Week 2 ready to discuss: which of the 4 strategies (Native/Hybrid/Cross-platform/PWA) fits your own project idea, and why.

Q&A